OmniGroom Platform

Multi-Tenant System Architecture & Detailed Requirements Blueprint

A complete production-ready specification for a multi-tenant booking, management, and marketing SaaS tailored for Salons, Barbershops, Day Spas, and Animal Grooming facilities.

Target Stack:	Supabase, Render, Vercel, Payfast, 1Voucher
Pricing Model:	Flat R65.00 ZAR / Month / Facility
Version:	1.0.0 (Production Specification)
Classification:	Confidential - Technical Architecture

1. EXECUTIVE & TECHNICAL ARCHITECTURE SUMMARY

The platform is a multi-tenant Software-as-a-Service (SaaS) engine built to optimize operational workflows for service-oriented businesses like hair salons, barbershops, high-end day spas, massage parlors, and pet grooming facilities. The architecture enforces an absolute distinction between physical locations and parent organizational ownership, ensuring that multi-facility owners possess a centralized workspace while ensuring strict data and subscription isolation per location.

The backend infrastructure utilizes **Supabase** for core identity management, data warehousing, and granular file storage, alongside an application layer deployed to **Render** designed to orchestrate long-running processes, real-time social webhooks, and third-party validation APIs. The customer acquisition storefront resides as a statically optimized application on **Vercel**.

> **Strategic Architecture Constraint:** Consumer billing for individual appointments is executed completely offline at the physical venues (e.g., physical point-of-sale systems, cash, or direct on-site transfers). Online ledger integrations are exclusively dedicated to handling the uniform platform access fee subscription of **R65.00 ZAR per month per facility**.

2. TECHNICAL INFRASTRUCTURE STACK

To ensure low operational overhead, near-instant scaling boundary definitions, and rapid data access, the system utilizes the following integrated cloud services:

Layer / System	Provider / Component	Functional Responsibility
Database Engine	Supabase (PostgreSQL)	Data persistence, native Row Level Security (RLS) enforcement, schema partitioning rules, and storage hooks.
Authentication Core	Supabase Auth	Handles user identity management via Mobile OTP, Google OAuth, and Facebook Identity pools.
Application Backend	Render (Node.js/Express)	Executes business scheduling logic, acts as webhook receiver for Meta APIs, handles daily cron maintenance, and interfaces with billing portals.
Web Marketing Core	Vercel (Next.js)	Hosts the public marketing landing page, performance-driven tracking, and handles initial account registration pipelines.
Mobile Engine	React Native via Expo	Powers dual application bundles downloadable via iOS and Android marketplaces (Client Booking App & Business Management App).
Communication Core	Twilio API	Orchestrates critical baseline alerts, authentication OTP tokens, and SMS notification structures.
Push Gateway	Expo Notification Engine	Free tier abstraction framework mapping structural signals directly into native Apple and Google push networks.
SaaS Payment Engine	Payfast & Flash 1Voucher	Processes digital recurring subscription invoices (Payfast) and checks prepaid physical voucher codes (1Voucher).

3. COMPLETE DATABASE SCHEMA CONFIGURATION

The entire relational ecosystem utilizes PostgreSQL features natively supported in Supabase. Row-Level Security policies are declared below to guarantee complete system isolation between tenants.

```
-- Core User Extensions Linked to Auth Identity
CREATE TABLE public.profiles (
    id UUID REFERENCES auth.users ON DELETE CASCADE PRIMARY KEY,
    phone_number VARCHAR(20) UNIQUE NOT NULL,
    full_name VARCHAR(100) NOT NULL,
    role VARCHAR(20) NOT NULL CHECK (role IN ('client', 'staff', 'owner', 'platform_admin')),
    created_at TIMESTAMPTZ DEFAULT now() NOT NULL
);

-- Global Platform Constraints Configuration
CREATE TABLE public.platform_config (
    id INT PRIMARY KEY DEFAULT 1 CHECK (id = 1),
    subscription_price_zar NUMERIC(10, 2) NOT NULL DEFAULT 65.00,
    max_gallery_images_per_facility INT NOT NULL DEFAULT 15,
    updated_at TIMESTAMPTZ DEFAULT now() NOT NULL
);

-- Parent Facility/Tenant Model
CREATE TABLE public.facilities (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    owner_id UUID REFERENCES public.profiles(id) ON DELETE RESTRICT NOT NULL,
    name VARCHAR(150) NOT NULL,
    facility_type VARCHAR(50) NOT NULL CHECK (facility_type IN ('salon', 'barber_shop',
'animal_grooming', 'day_spa', 'massage_parlor')),
    working_hours JSONB NOT NULL, -- Format: {"monday": {"open": "08:00", "close": "17:00",
"is_closed": false}, ...}
    created_at TIMESTAMPTZ DEFAULT now() NOT NULL
);

-- Subscription Status Tracker Matrix
CREATE TABLE public.facility_subscriptions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    facility_id UUID REFERENCES public.facilities(id) ON DELETE CASCADE UNIQUE NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'trial' CHECK (status IN ('trial', 'active',
'expired')),
    payment_method VARCHAR(20) CHECK (payment_method IN ('payfast', '1voucher')),
    current_period_end TIMESTAMPTZ NOT NULL DEFAULT (now() + interval '14 days'),
    last_payment_date TIMESTAMPTZ,
    last_gateway_reference VARCHAR(100)
);

-- Global Centralized Unaltered Catalog
CREATE TABLE public.service_catalog (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(150) NOT NULL,
    category VARCHAR(50) NOT NULL,
    default_placeholder_image VARCHAR(255) NOT NULL,
    created_at TIMESTAMPTZ DEFAULT now() NOT NULL
);

-- Facility-Specific Dynamic Catalogs (Overrides Global)
CREATE TABLE public.facility_services (
```

```
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    facility_id UUID REFERENCES public.facilities(id) ON DELETE CASCADE NOT NULL,
    catalog_id UUID REFERENCES public.service_catalog(id) ON DELETE RESTRICT NOT NULL,
    custom_name VARCHAR(150), -- Null values trigger fallback to global
    price_zar NUMERIC(10, 2) NOT NULL,
    duration_minutes INT NOT NULL,
    custom_image_url VARCHAR(255), -- Null triggers default_placeholder_image
    is_active BOOLEAN DEFAULT true NOT NULL,
    UNIQUE(facility_id, catalog_id)
);

-- Staff Resource Registries
CREATE TABLE public.staff (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    facility_id UUID REFERENCES public.facilities(id) ON DELETE CASCADE NOT NULL,
    user_id UUID REFERENCES public.profiles(id) ON DELETE RESTRICT NOT NULL,
    custom_working_hours JSONB, -- Overrides facility default if provided
    is_active BOOLEAN DEFAULT true NOT NULL,
    UNIQUE(facility_id, user_id)
);

-- Staff Capabilities Junction Registry
CREATE TABLE public.staff_service_skills (
    staff_id UUID REFERENCES public.staff(id) ON DELETE CASCADE NOT NULL,
    facility_service_id UUID REFERENCES public.facility_services(id) ON DELETE CASCADE NOT NULL,
    PRIMARY KEY (staff_id, facility_service_id)
);

-- Central Real-Time Appointment Engine
CREATE TABLE public.appointments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    facility_id UUID REFERENCES public.facilities(id) ON DELETE CASCADE NOT NULL,
    client_id UUID REFERENCES public.profiles(id) ON DELETE RESTRICT NOT NULL,
    staff_id UUID REFERENCES public.staff(id) ON DELETE RESTRICT NOT NULL,
    facility_service_id UUID REFERENCES public.facility_services(id) ON DELETE RESTRICT NOT
NULL,
    start_time TIMESTAMPTZ NOT NULL,
    end_time TIMESTAMPTZ NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'pending' CHECK (status IN ('pending', 'confirmed',
'completed', 'cancelled')),
    offline_payment_method VARCHAR(20) CHECK (offline_payment_method IN ('cash', 'card',
'eft')),
    created_at TIMESTAMPTZ DEFAULT now() NOT NULL
);

-- Marketing Portfolio Management
CREATE TABLE public.facility_gallery (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    facility_id UUID REFERENCES public.facilities(id) ON DELETE CASCADE NOT NULL,
    image_url VARCHAR(255) NOT NULL,
    uploaded_at TIMESTAMPTZ DEFAULT now() NOT NULL
);
```

```
-- Dynamic Promotional Offer Rules Matrix
CREATE TABLE public.promotions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    facility_id UUID REFERENCES public.facilities(id) ON DELETE CASCADE NOT NULL,
    promo_code VARCHAR(30) NOT NULL,
    discount_percentage INT NOT NULL CHECK (discount_percentage > 0 AND discount_percentage <=
100),
    start_date TIMESTAMPTZ NOT NULL,
    end_date TIMESTAMPTZ NOT NULL,
    is_active BOOLEAN DEFAULT true NOT NULL,
    UNIQUE(facility_id, promo_code)
);
```

Row Level Security (RLS) Policy Execution Model

Data visibility constraints must be completely offloaded to the database engine via RLS. Below is an example of an actual production policy applied to operational data to lock down multi-tenancy access paths securely:

```
-- Force Global Read Policies for Base Services
ALTER TABLE public.service_catalog ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Allow authenticated users global read access"
ON public.service_catalog FOR SELECT TO authenticated USING (true);

-- Enforce Multi-Tenant Separation Rules on Appointments
ALTER TABLE public.appointments ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Clients can view their own personal appointment timelines"
ON public.appointments FOR SELECT TO authenticated
USING (auth.uid() = client_id);

CREATE POLICY "Facility Owners can manage details within their tenants"
ON public.appointments FOR ALL TO authenticated
USING (
    facility_id IN (
        SELECT f.id FROM public.facilities f
        WHERE f.owner_id = auth.uid()
    )
);

CREATE POLICY "Assigned Staff can view details of scheduled duties"
ON public.appointments FOR SELECT TO authenticated
USING (
    staff_id IN (
        SELECT s.id FROM public.staff s
        WHERE s.user_id = auth.uid()
    )
);
```

4. FINANCIAL OPERATIONS & SUBSCRIPTION LIFECYCLE

The platform enforces a highly standardized recurring monthly charge of $P = 65.00 \ ext{ ZAR}$. To capture maximum target audiences within both formalized digital structures and unbanked/cash-reliant economies, the platform operates dual ingestion channels:

A. Payfast Processing Engine (Auto-Recurring)

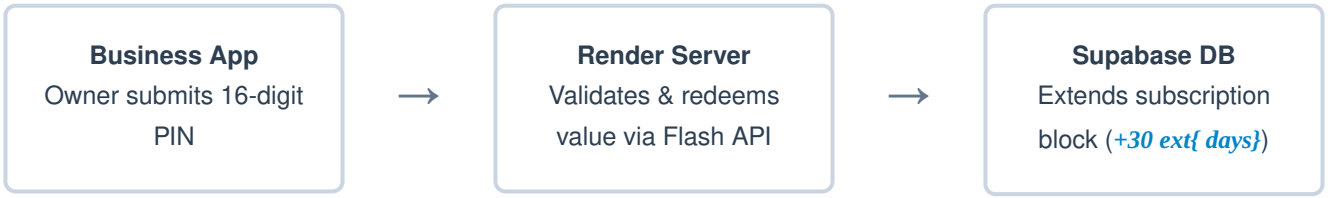
Digital payment instruments execute against the Payfast Ad Hoc API endpoints. During setup, the owner processes an initial authorization token.

- **Payload Receiver:** The backend exposes a secure target point at `POST /api/v1/payments/payfast-webhook` mapped on the Render cluster.
- **Verification Strategy:** The backend verifies the incoming IP against public Payfast outbound address lists, recalculates the cryptographic signature, and reads the `payment_status` parameter.
- **Database Action:** Upon confirmation of payment, the expiration window is updated:

$$T_{ \ ext{end}} = \max(T_{ \ ext{current\_end}}, T_{ \ ext{now}}) + 30 \ ext{ days}$$

B. Flash 1Voucher Validation Engine (Prepaid Cash System)

Owners buy physical 1Voucher slips at retail vendors. When they redeem a voucher, the mobile app sends the 16-digit voucher PIN to the backend, which forwards it to the Flash distribution engine for instant settlement.



- **Business Rule Rules & Multipliers:** The Flash execution API returns the numeric face-value currency score of the token. If the token value matches exactly $65.00 \ ext{ ZAR}$, the subscription extends by 30 days. If the owner inputs multi-month denominations, the system computes the exact extension matrix value via:

$$\Delta_{ \ ext{days}} = \left(rac{ \ ext{Voucher Value}}{65.00} ight) imes 30$$

  If the computation results in fractional anomalies or drops below the baseline requirement threshold ($ext{Value} < 65.00$), the API triggers a validation rejection code, leaving the voucher untouched.

C. Core Suspension Cron Framework

A background daemon schedule executes every night at midnight on Render to scan for lapsed accounts.

```
// Execution Script Target Logic running daily at 00:01:00 AM
UPDATE public.facility_subscriptions
SET status = 'expired'
WHERE current_period_end < now() AND status != 'expired';
```

When a business moves into an expired status, it is instantly filtered out from consumer visibility feeds in the search API, ensuring that only active tenants can receive user bookings.

5. DYNAMIC BOOKING & CONSTRAINTS ENGINE

The availability matrix is calculated in real time whenever a client selects a service. The scheduling engine determines open time slots by combining several data parameters into a cohesive availability window:

AVAILABILITY CALCULATION EQUATION

$$\text{Available Slots} = (\text{Facility Operating Hours} \cap \text{Staff Working Hours}) \setminus (\text{Existing Confirmed Appointments} \cup \text{Service Buffer Times})$$

Operational Rules for Media Upload Limits

To keep data storage predictable and control cloud hosting costs, the system enforces a strict image limit across all facilities. The system pulls the `max_gallery_images_per_facility` count variable from the global configuration table during upload attempts.

When a business manager uploads a new promotional asset, the Render node server queries the active gallery file counts. If the current total matches or exceeds the configuration limit, the asset upload is rejected before writing to storage, and an error message is returned to the user interface.

6. OMNICHANNEL META MESSAGING GATEWAY

The communication layer builds a centralized support experience by streaming direct user interactions from external Meta social platforms straight into the local mobile business app dashboard.

Data Ingestion Integration Sequence

1. **The Inbound Event:** A potential client sends a direct message to a facility's registered Instagram Business profile or Facebook Page.
2. **Webhook Dispatch:** Meta's Graph API pushes an encrypted JSON payload tracking this event to the platform's public integration endpoint on Render: `POST /api/v1/integrations/meta-webhook`.
3. **Payload Verification:** The backend verifies the message payload signature using the registered `X-Hub-Signature-256` secret header.

4. **Tenant Mapping:** The ingestion parser maps the incoming platform page identifier code to its corresponding `facility_id` record in the database.
5. **Persistent Logging:** The text content is appended to a specialized messaging history table, creating or updating a unified customer thread.
6. **Push Alert Routing:** The system sends a notification via the Expo Push Gateway, routing the message alert to any active mobile devices used by the tenant's staff or business owners.

7. PLATFORM WEB FRONT-END ARCHITECTURE

The public web interface is built using Next.js on Vercel to optimize search discovery, ensure fast page loads, and act as the primary registration portal for new business owners.

Functional User Interface Blueprint

- **Hero Value Statement Panel:** Displays clear value propositions tailored to each supported industry, accompanied by prominent App Store and Google Play download badges.
- **Real-Time Registration Engine Component:** A lightweight account creation workflow that captures the business owner's registration details, verifies their phone identity via OTP, and writes their primary user profile into Supabase Auth. This allows owners to log straight into the mobile business app instantly.
- **Flat-Rate Pricing Interface:** A clean, transparent pricing card explicitly highlighting the flat-rate fee structure: *“R65.00 ZAR per month per location. Full feature access. No hidden processing tiers.”*

Design Guidelines for Business Operations Tools

The administrative dashboards handle complex data grids, multi-branch scheduling interfaces, calendar matrix views, and rapid user profile switching. To maximize text readability and limit visual fatigue during long operational shifts, the interface strictly relies on solid background tones (such as pure white and slate gray). Gradients, distracting image patterns, and heavy UI card styles are explicitly banned within workspace components to maintain a clean, readable layout.